```
# !pip install pymupdf unstructured sentence-transformers pinecone groq tqdm
# !pip install pymupdf
```

```
import fitz
import json
import uuid
from tqdm import tqdm

from unstructured.cleaners.core import clean_extra_whitespace
from unstructured.cleaners.core import clean
from sentence_transformers import SentenceTransformer

from pinecone import Pinecone, ServerlessSpec

from groq import Groq
```

```
PDF_PATH = "input.pdf"
# !pip install crewai
import crewai
INDEX_NAME = "dsa-concepts-index"

GROQ_API_KEY = "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"
PINECONE_API_KEY = "pcsk_5UZRrZ_UxTydFzrYVSUZgH1YJHXheYb8nSqEgWhnr3P5nMWJzbT1vUb2y99uusHQ91NAC3"

EMBED_MODEL = "all-MiniLM-L6-v2"
```

```
import os
import fitz
import uuid
import json
from tqdm import tqdm
from dotenv import load_dotenv
from sentence_transformers import SentenceTransformer
from pinecone import Pinecone, ServerlessSpec
from groq import Groq
import time
```

```
load_dotenv()

PDF_PATH = "input.pdf"
# !pip install crewai
import crewai
INDEX_NAME = "dsa-concepts-index"

GROQ_API_KEY = "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"
PINECONE_API_KEY = "pcsk_5UZRrZ_UxTydFzrYVSUZgH1YJHXheYb8nSqEgWhnr3P5nMWJzbT1vUb2y99uusHQ91NAC3"

pc = Pinecone(api_key=PINECONE_API_KEY)
client = Groq(api_key=GROQ_API_KEY)
EMBED_MODEL = "all-MiniLM-L6-v2"

model = SentenceTransformer(EMBED_MODEL)
```

```
def extract_pdf(pdf_path):

    doc = fitz.open(pdf_path)

    pages = []

    for page_num, page in enumerate(doc):

        text = page.get_text()

        if text.strip():
            pages.append({
                "page": page_num,
                "text": text
```

```
            })

    return pages
```

```
def chunk_text(pages, chunk_size=1000, overlap=200):

    chunks = []

    for page in pages:

        text = page["text"]

        start = 0

        while start < len(text):

            chunk = text[start:start+chunk_size]

            chunks.append({
                "id": str(uuid.uuid4()),
                "text": chunk,
                "page": page["page"]
            })

            start += chunk_size - overlap

    return chunks
```

```
def create_embeddings(chunks):

    for chunk in tqdm(chunks):

        embedding = model.encode(chunk["text"]).tolist()

        chunk["embedding"] = embedding

    return chunks
```

```
def store_pinecone(chunks):

    existing = [i["name"] for i in pc.list_indexes()]

    if INDEX_NAME not in existing:

        pc.create_index(
            name=INDEX_NAME,
            dimension=384,
            metric="cosine",
            spec=ServerlessSpec(
                cloud="aws",
                region="us-east-1"
            )
        )

    index = pc.Index(INDEX_NAME)

    vectors = []

    for chunk in chunks:

        vectors.append({
            "id": chunk["id"],
            "values": chunk["embedding"],
            "metadata": {
                "text": chunk["text"],
                "page": chunk["page"]
            }
        })

    index.upsert(vectors=vectors)
```

```
    print("Stored in Pinecone")
```

```
def extract_chunk(chunk):

    prompt = f"""
Extract ALL structured department knowledge from this text.

Return STRICT VALID JSON only.

Do NOT include explanations.
Do NOT include markdown.
Do NOT include ```json

Return ONLY raw JSON.

Structure:

{{
  "staff": [],
  "equipment": [],
  "materials": [],
  "contractors": [],
  "projects": [],
  "budget": [],
  "zones": [],
  "grievances": [],
  "requests": [],
  "sla": [],
  "kpis": [],
  "maintenance": [],
  "alerts": [],
  "ai_insights": [],
  "officers": [],
  "performance": [],
  "inventory": []
}}

TEXT:
{chunk["text"]}
"""

    try:

        completion = client.chat.completions.create(

            model="llama-3.3-70b-versatile",

            temperature=0,

            messages=[
                {
                    "role": "system",
                    "content": "You extract structured data and return ONLY valid JSON."
                },
                {
                    "role": "user",
                    "content": prompt
                }
            ]

        )

        result = completion.choices[0].message.content

        if not result:
            print("Empty response")
            return None

        # Remove markdown if exists
        result = result.strip()

        if "```json" in result:
            result = result.split("```json")[1].split("```")[0]
```

```
        elif "```" in result:
            result = result.split("```")[1]

        result = result.strip()

        # Find first JSON bracket
        start = result.find("{")
        end = result.rfind("}") + 1

        if start == -1 or end == -1:
            print("No JSON found")
            return None

        result = result[start:end]

        return json.loads(result)

    except json.JSONDecodeError as e:

        print("JSON decode error")
        print(result[:500])
        return None

    except Exception as e:

        print("Extraction error:", e)
        return None
```

```
def merge_json(master, new):

    for key in master:

        if key in new and isinstance(new[key], list):

            master[key].extend(new[key])

    return master
```

```
def create_master():

    return {
        "staff": [],
        "equipment": [],
        "materials": [],
        "contractors": [],
        "projects": [],
        "budget": [],
        "zones": [],
        "grievances": [],
        "requests": [],
        "sla": [],
        "kpis": [],
        "maintenance": [],
        "alerts": [],
        "ai_insights": [],
        "officers": [],
        "performance": [],
        "inventory": []
    }
```

```
def build_knowledge_base(chunks):

    master = create_master()

    for chunk in tqdm(chunks):

        extracted = extract_chunk(chunk)

        if extracted:

            master = merge_json(master, extracted)

            with open("knowledge_base.json", "w") as f:
```

```
                    json.dump(master, f, indent=2)

            time.sleep(0.5)

        return master
```

```
def main():

    print("Extracting PDF...")
    pages = extract_pdf(PDF_PATH)

    print("Chunking...")
    chunks = chunk_text(pages)

    print("Embedding...")
    chunks = create_embeddings(chunks)

    print("Storing...")
    store_pinecone(chunks)

    print("Extracting structured knowledge...")
    knowledge = build_knowledge_base(chunks)

    print("Knowledge base built")



if __name__ == "__main__":
    main()
```

```
Extracting PDF...
Chunking...
Embedding...
100%|██████████| 47/47 [00:07<00:00,  5.96it/s]
Storing...
Stored in Pinecone
Extracting structured knowledge...
100%|██████████| 47/47 [00:45<00:00,  1.04it/s]Knowledge base built
```

```
def chunk_text(pages, chunk_size=1000, overlap=200):

    chunks = []

    for page in pages:

        text = page["text"]
        start = 0

        while start < len(text):

            chunk = text[start:start+chunk_size]

            chunks.append({
                "id": str(uuid.uuid4()),
                "text": chunk,
                "page": page["page"]
            })

            start += chunk_size - overlap

    return chunks
```

```
def create_embeddings(chunks):

    for chunk in tqdm(chunks, desc="Embedding"):

        embedding = model.encode(chunk["text"]).tolist()

        chunk["embedding"] = embedding

    return chunks
```

```
def extract_pdf_text(path):

    doc = fitz.open(path)
    pages = []

    for page_num, page in enumerate(doc):

        text = page.get_text()

        if text.strip():

            pages.append({
                "page": page_num,
                "text": text
            })

    return pages
```

```
def store_in_pinecone(chunks):

    existing_indexes = [i["name"] for i in pc.list_indexes()]

    if INDEX_NAME not in existing_indexes:

        pc.create_index(
            name=INDEX_NAME,
            dimension=384,
            metric="cosine",
            spec=ServerlessSpec(
                cloud="aws",
                region="us-east-1"
            )
        )

        print("Created Pinecone index")

    index = pc.Index(INDEX_NAME)

    vectors = []

    for chunk in chunks:

        vectors.append({
            "id": chunk["id"],
            "values": chunk["embedding"],
            "metadata": {
                "text": chunk["text"],
                "page": chunk["page"]
            }
        })

    index.upsert(vectors=vectors)

    print("Stored in Pinecone")
```

```
def extract_chunk_universal(text):

#     prompt = f"""
# Convert this text into structured JSON.

# Extract EVERYTHING useful.

# Rules:
# - Create keys dynamically
# - No fixed schema
# - Capture entities, numbers, dates, people, projects
# - Use nested JSON if needed
# - Return ONLY valid JSON

# TEXT:
# {text}
# """
    prompt = f"""
    Convert the given text into CLEAN, MEANINGFUL, DASHBOARD-READY JSON.
```

```
CRITICAL RULES:

1. Use only meaningful keys like:
  - department
  - projects
  - officials
  - services
  - schemes
  - locations
  - contacts
  - programs

2. REMOVE useless data like:
  - random numbers
  - serial numbers
  - IDs without meaning
  - duplicate entries
  - empty fields

3. DO NOT create keys like:
  numbers, codes, ids, indexes, sr_no

4. Always group into logical categories:
  projects → all missions, schemes
  officials → officers, engineers
  services → public services
  department → department info

5. Make dashboard-friendly structure.

6. Return ONLY valid JSON.

TEXT:
{text}
"""

try:

    completion = client.chat.completions.create(

        model="llama-3.3-70b-versatile",

        temperature=0,

        messages=[
            {
                "role": "system",
                "content": "You convert unstructured text into structured JSON knowledge."
            },
            {
                "role": "user",
                "content": prompt
            }
        ]

    )

    result = completion.choices[0].message.content.strip()

    if not result:
        return None

    if "```" in result:

        parts = result.split("```")

        for part in parts:
            if "{" in part:
                result = part
                break

    start = result.find("{")
    end = result.rfind("}") + 1

    result = result[start:end]

    return json.loads(result)
```

```
    except Exception as e:

        print("Extraction error:", e)
        return None
```

```
def merge_json(master, new):

    if not new:
        return master

    for key, value in new.items():

        if key not in master:

            master[key] = value

        else:

            # CASE 1: both dict → merge dict
            if isinstance(master[key], dict) and isinstance(value, dict):

                master[key].update(value)

            # CASE 2: both list → extend list
            elif isinstance(master[key], list) and isinstance(value, list):

                master[key].extend(value)

            # CASE 3: master list, new single → append
            elif isinstance(master[key], list):

                master[key].append(value)

            # CASE 4: master single, new list → convert to list
            elif isinstance(value, list):

                master[key] = [master[key]] + value

            # CASE 5: both single values → convert to list
            else:

                master[key] = [master[key], value]

    return master
```

```
def build_knowledge_base(chunks):

    master = {}

    for chunk in tqdm(chunks, desc="Extracting knowledge"):

        extracted = None

        for attempt in range(3):

            extracted = extract_chunk_universal(chunk["text"])

            if extracted:
                break

            time.sleep(1)

        master = merge_json(master, extracted)

        with open("knowledge_base.json", "w", encoding="utf-8") as f:

            json.dump(master, f, indent=2, ensure_ascii=False)

    return master
```

```
def main():
```

```
    print("Extracting PDF...")
    pages = extract_pdf_text(PDF_PATH)

    print("Chunking...")
    chunks = chunk_text(pages)

    print("Embedding...")
    chunks = create_embeddings(chunks)

    print("Storing in Pinecone...")
    store_in_pinecone(chunks)

    print("Building dynamic knowledge base...")
    build_knowledge_base(chunks)

    print("DONE. knowledge_base.json created.")


if __name__ == "__main__":
    main()
```